



Spring Semester, 2022

CSE 211 – Introduction to Embedded

Lab. 2: C Build Process

Goals of this Lab:

1- Know the importance of build process. 2- Understand the build process stages.

What is the “C build process”?

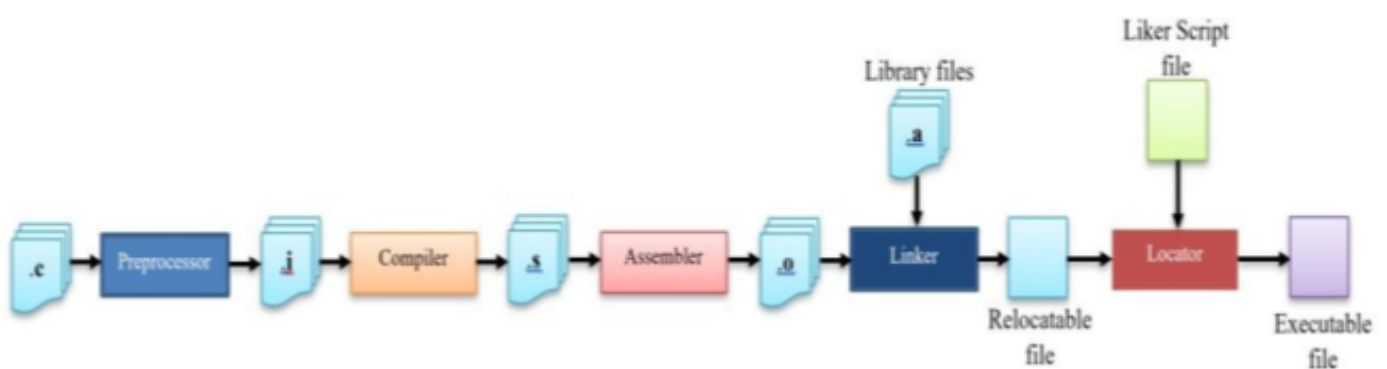
C Build Process is considered one of the most important topics in the field of Embedded Software and it's the most famous question you will be asked in any Embedded Software interview.

Today we will talk about each step of this process in details to help you get a better understanding of this process.

C Build Process is the process of converting the high-level source code representation of your embedded software into an executable binary image. This Process involves many steps and tools, but the main three distinct steps of this process are:

- Each of the source files must be compiled or assembled into an object file.
- All the object files that result from the first step must be linked together to produce a single object file, called the relocatable program.
- Physical memory addresses must be assigned to the relative offsets within the relocatable program in a process called relocation.

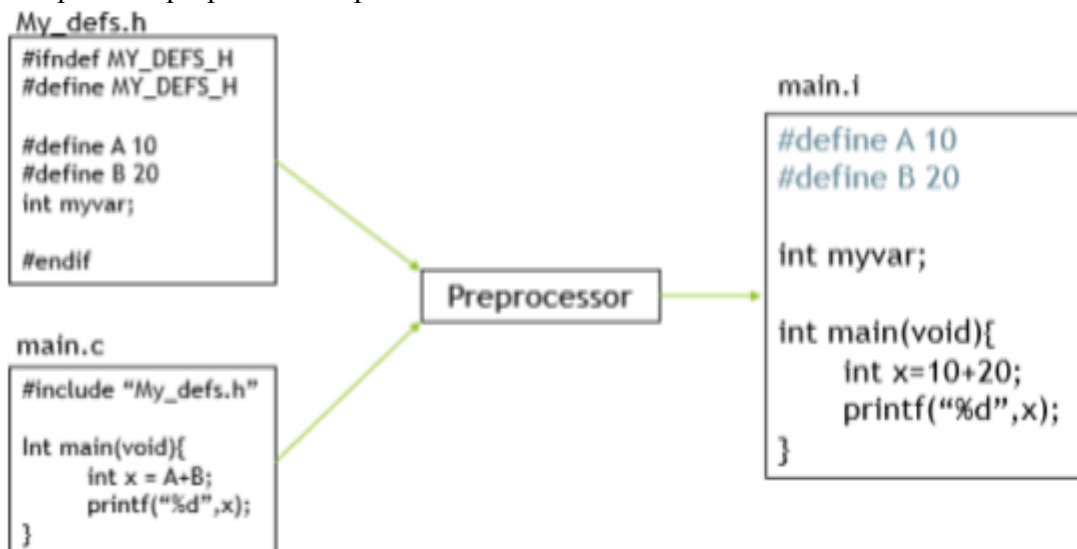
The result of the final step is a file containing an executable binary image that is ready to run on the embedded system.



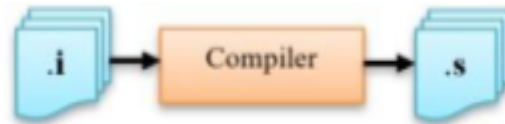
1. Preprocessor



- The input file for this stage is *.c file.
- The output file is *.i or preprocessed file.
- The preprocessor provides a special set of keywords called “preprocessor directives” and they are keywords preceded by the “#” symbol. The preprocessor strips out comments from the input c file. Evaluate preprocessor directive by making substitution for lines started with #, and then produces a pure C code without any preprocessor directives.
- Note that if a bug/error happened in the preprocessor stage you normally won’t know its place as the output of the preprocessor goes directly into compiler, the error will be likely at the lines you used the preprocessor directive.
- Example on a preprocessor operation:



2. Compiler



- In this stage the C code gets converted into architecture specific **assembly** by the compiler; this conversion is **not a one to one mapping of lines** but instead a decomposition of C operations into numerous assembly operations. Each operation itself is a **very basic task**.
- The input file for this stage is *.i file.
- The output file is *.s or *.asm file.
- The compiler works with each translation unit at a time. It checks the syntax correctness of each file **separately**.
- An example of a compiler is the GNU's C compiler known as "gcc".
- The compiler can be **native compiler** or **cross compiler**.
- Example on the compiler:

C-Programming (High Level Language)

```
int x = 0;
int y = 20;
int z = 5;
...
while (y >= z) {
    y = y - z;
    x++;
}
```

ARM Assembly Language

```
ldr    r2, y
ldr    r3, z
ldr    r4, x
LOOP:
sub    r2, r3
inc    r4
cmp    r2, r3
bgt    LOOP
str    r2, y
str    r4, x
```

Native Compiler vs Cross Compiler

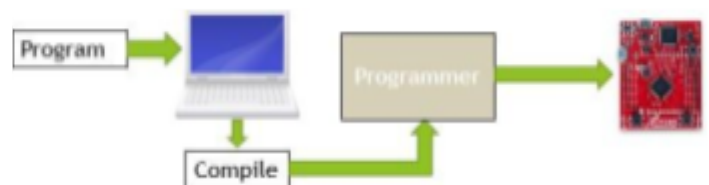
Native Compiler

- A native compiler is a compiler that compiles the program and produces a final executable that is executed on **the machine on which the compiler runs**.
- For example, you write a program on your laptop or PC and compile it and run it on your laptop or PC so the executable object file that is produced is **dependent on your laptop or PC architecture**.



Cross Compiler

- A cross compiler on the other hand is a compiler that compiles the program and produces a final executable that is intended to **run on a target machine**.
- For example, you write a program on your laptop or PC and compile it but the generated executable file is intended to run on a target board that you have (Tiva C for example) and hence this executable file is **dependent on your board**.



3. Assembler



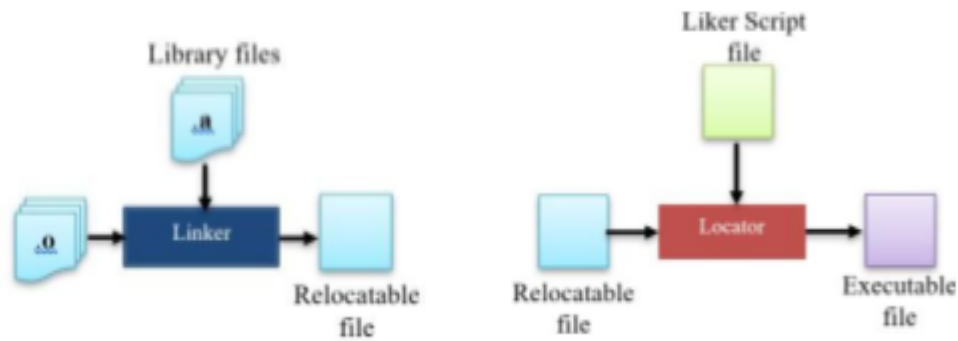
- The assembler converts the assembly code files (.s files) to object files (.o files). It performs one to one translation of assembly instructions to machine instructions that can actually run on the target.
- The input to the assembler is assembly code files (.s files or .asm files).
- The output is object code files (.o files or .obj files).
- Machine instructions are a bunch of ones and zeros; hence they are not human readable.
- We sometimes refer to the compiler + assembler as the compiler.
- Example on the assembler:



Assembly Language	Machine Code
movs r3, #0	0x2300
strb r3, [r7, #7]	0x71fb
adds r3, r7, #7	0x1dfb
str r3, [r7, #8]	0x60bb
adds r1, r7, #7	0x1df9
adds r3, r7, #6	0x1dbb

Target dependant

4. Linker and Locator



- The linker is the **most important part** of the compilation environment.
- Inputs to the linker are:
 - Object files produced by the compiler
 - Standard libraries
 - Linker script
- The output of the linker is a .bin file which is the final executable image that can run on the target.
- .bin file takes many forms such as .elf, .hex, .out ...etc.
- The linker merges the object files in order to produce a final executable image which can run on the target.
- The linker's resolves all the external data and functions of each object file and performs "linking" between them.
- The linker allocates the target memory and then the locator which is the part of linker responsible for memory allocation gives addresses to the variables and functions.
- The linker also generates a .map file which contains the allocation of each object to the memory.
- Notes:
 - The locator can be found as a separate tool or with the linker.
 - Only the .bin file is burnt and run on the target the .map doesn't get burnt on the target.
 - The .map file is linker type dependent.

5. Linker File Script

- Linker Script File or linker configuration file is responsible to tell the locator how to map the executable into proper addresses.
- It controls the memory layout of the output file as it contains the **memory mapping information of the segments of memory of the target**.
- The linker file can be used to change the location of a memory segment. For example, we can use it to change the address of the .data section in memory of the target or to create our own memory segment.
- From the info. provided in it and with the use of “**#pragma**” directive we can change the default segment of a specific variable and place it into another segment.
- The linker file **sets the general memory layout for the target**, that is it sets the addresses for the memory segments of the target memory.
- Example on linker file command

```
/*
 *
 * Default Linker Command file for the Texas Instruments TM4C123GH6PM
 *
 * This is derived from revision 15071 of the TivaWare Library.
 *
 */

--retain=g_pfnVectors

MEMORY
{
    FLASH (RX) : origin = 0x00000000, length = 0x00040000
    SRAM (RWX) : origin = 0x20000000, length = 0x00080000
}

/* The following command line options are set as part of the CCS project. */
/* If you are building using the command line, or for some reason want to */
/* define them here, you can uncomment and modify these lines as needed. */
/* If you are using CCS for building, it is probably better to make any such */
/* modifications in your CCS project and leave this file alone. */
/*
/* --heap_size=0
/* --stack_size=256
/* --library=rtsv7M4_T_le_eabi.lib

/* Section allocation in memory */

SECTIONS
{
    .intvecs: > 0x00000000
    .text : > FLASH
    .const : > FLASH
    .cinit : > FLASH
    .pinit : > FLASH
    .init_array : > FLASH

    .vtable : > 0x20000000
    .data : > SRAM
    .bss : > SRAM
    .sysmem : > SRAM
    .stack : > SRAM
}

__STACK_TOP = __stack + 512;
```

6. Linking Types

- As illustrated previously, the linker performs linking between **the external variables and functions and their definitions**.
 - Having calls to functions in libraries there are two possible ways to perform the linking, **static** linking and **dynamic** linking.
 - Static linking is the process of **copying the used modules from libraries** to the final executable image so that the program is a whole entity represented by this executable image.
 - Files that use statically linked libraries are **larger in size** than those which use shared libraries, but they are **faster**.
 - Dynamic linking on the other hand **uses “references” to the called modules from a library** to resolve the external references and hence the files that use dynamically linked libraries are **smaller in size** than those that use statically linked libraries, but they are **slower**.
 - In case of static linking, if the called modules in libraries are modified, we need to **recompile and relink** in order to have the changes made to these modules reflected to our program.
 - On the other hand, in dynamic linking we **do not need to recompile or relink** even if the called module are modified since we use references to them, so we access them on each call.
-

7. Startup Code

- The startup code is a small code written in “assembly” language which runs after powerup and any kind of reset and it performs the basic startup initialization required before running the program.
- It contains the code to initialize the variables that the software needs to run correctly, e.g. if your program has the line `int x=5;` you want “x” to be equal to 5 when the program starts running so the startup code initializes “x” to 5 before the program runs.
- The startup code main tasks are:
 1. Disable all interrupts.
 2. Clock initialization and stabilization.
 3. Memory initialization:
 - Initialize the H/W memory mapped registers.
 - Copy any initialized data from the flash to the data segment in RAM.
 - Set the uninitialized variables in the .bss segment to zero.
 - Allocate space for and initialize the stack.
 4. Initialize the processor’s stack pointer to point to the end of stack.
 5. Call `main()`.

8. File System

- What should be added in the .c files ?
 - Function definitions
 - Variable declarations
 - #defines that need to be seen only inside this .c files.
 - What should be in the .h files ?
 - Function prototypes
 - extern to variables so that other files could use the variable.
-

Lab B exercise:

Task1:

- Create a file called types.h where you will define all variables data types with different acronyms where the variable name will be as follows: (int+size in bits) add (u) at the beginning if the data type is unsigned and add (_ptr) at the end if the type is a pointer. Examples:
 - char as int8
 - unsigned char as uint8
 - int* as int32_ptr
 - unsigned int* as uint32_ptr
 - Follow the same sequence with all data types for the Tiva 32 bit microcontroller
 - Use the types.h file in all the tasks that will follow in this lab and the next ones.
- What is the difference between using the typedef for type definition and using the #define for the same cause ?

Task 2:

- define X as a 10 using the #define macro definition. Create also a global variable that is an int y. Then write down the differences between the X and y in the following points:
 - When are they resolved in the build process?
 - Where is each one of them stored?
 - How can each of them be used in the code? Which of them could be altered and why?

Lab Report(individual to be submitted one day before next Lab B)

Task 1:

- Create a macro that adds two parameters. Also Create a function that adds two variables and returns the sum. Then Compare the differences between the macro and the function in the following points:
 - When will each be resolved in the build process?
 - What will happen when you pass wrong parameters to each one of them?
 - What will be the sequence in the function call and how will that differ in the macro like function?
- What will happen if the variable is declared in the .h file ? Then the .h file is included in 2 .c files? Why do we add the variable extern declaration in the .h file ?

Task 2:

- What are the types of building errors ? Give as many examples as you can to each type with C code snapshots of these errors.
 - For example :
 - Type : Compiler Error
 - Example: Missing a semicolon
 - Code example: `myvar = 10`
-